

Руководства

Быстрый запуск Речевого движка

Добавьте голосовое управление в свой чат-бот с помощью ElevenLabs SDK.

В этом руководстве вы узнаете, как создать голосового помощника с помощью Speech Engine. Вы настраиваете сервер, который подключает вашу языковую модель к ElevenLabs, а затем подключаете браузерный клиент, чтобы пользователи могли вести голосовые диалоги с вашим помощником.

★ Используйте [навык Speech Engine от ElevenLabs](#), чтобы добавить голос в свой чат-бот:

```
$ npx skills add elevenlabs/skills --skill speech-engine
```

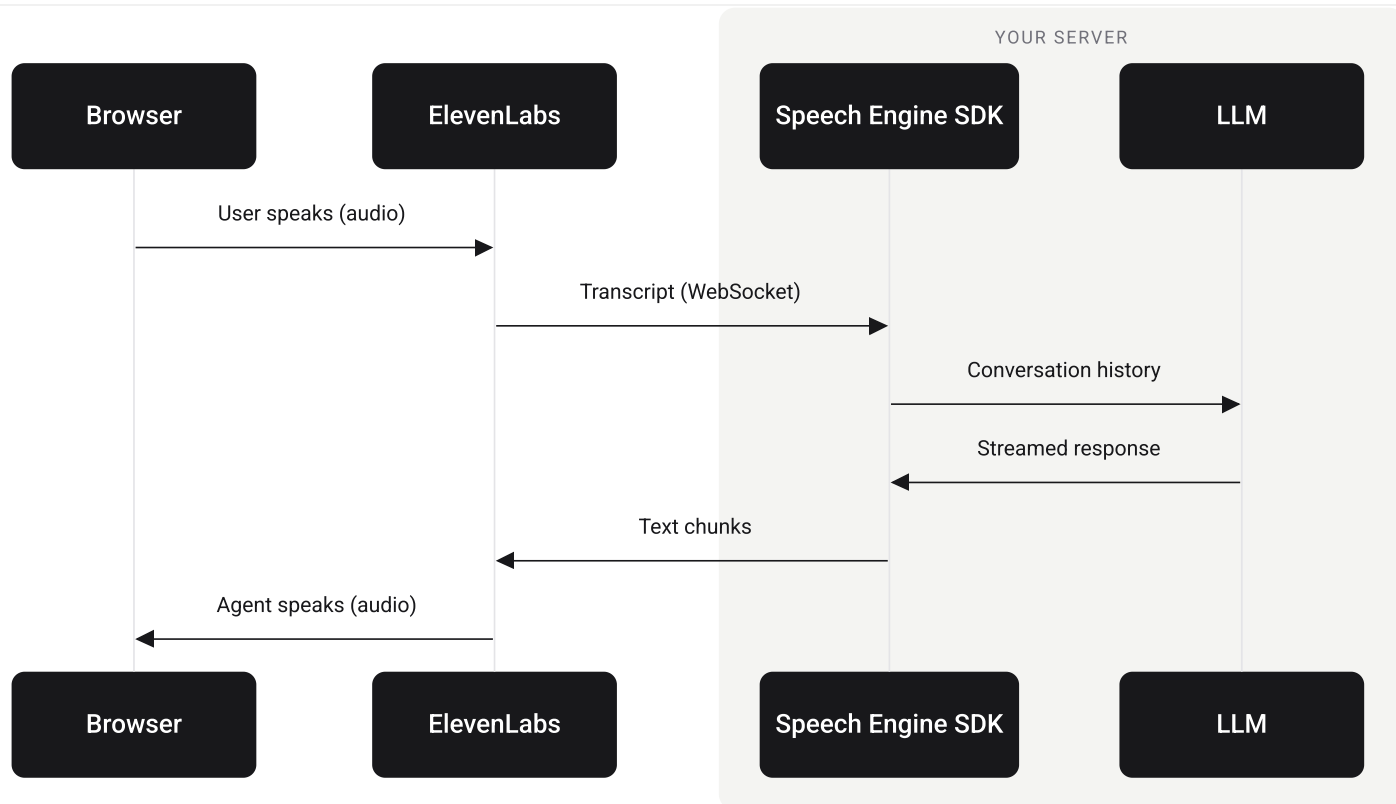
Как работает Speech Engine

Speech Engine подключает вашу языковую модель к ElevenLabs, чтобы она могла общаться с вашим ботом и слышать его ответы. ElevenLabs обрабатывает преобразование речи в текст и обратно в голос.



Начать звонок





Каждое соединение WebSocket представляет собой один разговор. Когда пользователь говорит, ElevenLabs расшифровывает аудио и отправляет расшифровку на ваш сервер. Ваш сервер передает ее в вашу языковую модель, а затем отправляет ответ обратно. ElevenLabs преобразует текст в речь и воспроизводит его в браузере. SDK управляет соединением, очередностью реплик и распознаванием прерывания разговора.

Предварительные требования

В этом руководстве используется API OpenAI для больших языковых моделей. Вам понадобится ключ API OpenAI, указанный в переменной `OPENAI_API_KEY` окружения.

Настройка сервера

1

Создать ключ API

Создайте ключ API на панели управления [здесь](#), который вы будете использовать для безопасного доступа к API.

Сохраните ключ как управляемый секрет и передайте его в SDK либо в качестве переменной среды через файл `.env` или непосредственно в коде. Выбор зависит от ваших предпочтений.

```
1 ELEVENLABS_API_KEY=<ваш_ключ_api_здесь>
```

2 Установить зависимости

Python

Машинопись

```
1 pip install elevenlabs openai python-dotenv
```

3 Сделать сервер общедоступным

Для Speech Engine нужен общедоступный URL. Используйте [ngrok](#), чтобы сделать локальный сервер общедоступным. Сервер еще не создан, но сначала нужно запустить ngrok, чтобы получить URL для следующего шага.

```
$ ngrok http 3001
```

Скопируйте URL для переадресации (например, `https://abc123.ngrok.io`).

4 Создайте экземпляр Speech Engine

С помощью SDK создайте экземпляр Speech Engine, передав URL-адрес ngrok с добавленным в качестве URL-адреса WebSocket путем `/ws`.

create_engine.py

create-engine.mts

```
1 import asyncio
2 from dotenv import load_dotenv
3 from elevenlabs import AsyncElevenLabs
4
5 load_dotenv()
6
7 elevenlabs = AsyncElevenLabs(
8     api_key=os.getenv("ELEVENLABS_API_KEY"),
9 )
10
11
12 async def main():
13     engine = await elevenlabs.speech_engine.create(
14         name="My Speech Engine",
15         speech_engine={
16             # Note we use the wss protocol instead of https
17             "ws_url": "wss://abc123.ngrok.io/ws",
```

20

21 `print(f"Speech Engine ID: {engine_engine_id}")`

Run this script and copy the Speech Engine ID (e.g. `seng_8k3m9xr4hjnfg983brhmhkd98n6`) for the next step.

5 Create the server

Create a file called `server.py` or `server.mts` with the following contents. This sets up a server, attaches Speech Engine on the `/ws` path, and uses OpenAI to generate responses.

`server.py`

`server.mts`

```
1  import asyncio
2  import os
3
4  from dotenv import load_dotenv
5  from openai import AsyncOpenAI
6  from elevenlabs import AsyncElevenLabs
7
8  load_dotenv()
9
10 # Replace with your Speech Engine ID from step 4
11 SPEECH_ENGINE_ID = "seng_8k3m9xr4hjnfg983brhmhkd98n6"
12
13 openai = AsyncOpenAI(
14     api_key=os.getenv("OPENAI_API_KEY"),
15 )
16 elevenlabs = AsyncElevenLabs(
17     api_key=os.getenv("ELEVENLABS_API_KEY"),
18 )
19
20
21 def on_init(conversation_id, session):
22     print(f"Session started: {conversation_id}")
23
24
25 async def on_transcript(transcript, session):
26     stream = await openai.responses.create(
27         model="gpt-4o",
28         instructions="You are a helpful voice assistant. Keep responses concise and use a friendly tone.",
29         input=[
30             {"role": "assistant" if transcript.startswith("A:") else "user", "content": transcript.split("A:")[1] if transcript.startswith("A:") else transcript},
31             for m in transcript
32         ],
33         stream=True,
```

```

36     await session.send_response(stream)
37
38
39 def on_close(session):
40     print(f"Session ended: {session.conversation_id}")
41
42
43 def on_error(err, session):
44     print(f"Error: {err}")
45
46
47 async def main():
48     engine = await elevenlabs.speech_engine.get(SPEECH_ENGINE_ID)
49
50     await engine.serve(
51         port=3001,
52         path="/ws",
53         debug=True,
54         on_init=on_init,
55         on_transcript=on_transcript,
56         on_close=on_close,
57         on_error=on_error,
58     )
59
60
61 if __name__ == "__main__":
62     asyncio.run(main())

```

The `onTranscript` / `on_transcript` callback receives the full conversation history and the current session. The TypeScript SDK also provides an `AbortSignal` that fires if the user interrupts mid-response. Passing `signal` to the OpenAI call cancels the LLM request automatically on interruption.

`sendResponse()` / `send_response()` accepts a string, an async iterable, or a stream from OpenAI, Anthropic, or Google Gemini. The SDK extracts the text content automatically.



In the above example, the full transcript from the user is passed to the LLM. In a production environment you should add guardrails to prevent any prompt injection or manipulation attempts.

6

Start the server

Python

TypeScript

```
1 python server.py
```

1 Install the client SDK

React JavaScript

```
$ npm install @elevenlabs/react
```

2 Create a token endpoint

Add a server-side endpoint that generates a conversation token. This keeps your API key out of the browser and uses WebRTC for the best audio quality.

token_server.py

token-server.mts

```
1 import os
2
3 from dotenv import load_dotenv
4 from flask import Flask, jsonify
5 from elevenlabs import ElevenLabs
6
7 load_dotenv()
8
9 app = Flask(__name__)
10 elevenlabs = ElevenLabs(
11     api_key=os.getenv("ELEVENLABS_API_KEY"),
12 )
13
14
15 @app.route("/api/token")
16 def get_token():
17     # Replace with your Speech Engine ID from step 4 of the server setup
18     speech_engine_id = "seng_8k3m9xr4hjnfg983brhmhkd98n6"
19
20     response = elevenlabs.conversational_ai.conversations.get_webrtc_token(
21         agent_id=speech_engine_id
```

3 Build the conversation UI

Fetch the conversation token from your server and use it to start a session.

React JavaScript

App.tsx

```
3
4  async function getToken(): Promise<string> {
5      const response = await fetch("/api/token");
6      if (!response.ok) {
7          throw Error("Failed to get conversation token");
8      }
9      const data = await response.json();
10     return data.token;
11 }
12
13 export default function App() {
14     const conversation = useConversation({
15         onConnect: () => console.log("Connected"),
16         onDisconnect: () => console.log("Disconnected"),
17         onError: (error: Error) => console.error("Error:", error),
18     });
19
20     const startConversation = useCallback(async () => {
21         await navigator.mediaDevices.getUserMedia({ audio: true });
```

4 Try it out

Make sure three processes are running:

- 1. **ngrok** - forwarding to port 3001
- 2. **Your Speech Engine server** - `python server.py` or `npx tsx server.mts`
- 3. **The token server** - `npx tsx token-server.mts` or `python token_server.py`

Open your client application in the browser and click **Start conversation**. Grant microphone access when prompted, then speak. You should hear the agent respond through your speakers.

If you have `debug: true` enabled on the server, you will see incoming transcripts and outgoing responses logged to the console.

Session events

Event	TypeScript callback	Python callback	Description
user_transcript	onTranscript	on_transcript	When the user speaks, the transcript is sent to the server. The transcript is a string of text representing the user's speech.
init	onInit	on_init	
			Session initialized with a conversation ID.

close

onClose

on_close

Clean disconnect from ElevenLabs.

Configuring the first agent message

By default, the agent waits for the user to speak first. To have the agent greet the user when the conversation starts, set a first message in the `overrides` option on the client when starting the session.

- 1 To allow the agent to speak first, we need to update the Speech Engine resource to allow setting this from the client.

Python

TypeScript



```
1 engine = await elevenlabs.speech_engine.update(
2     speech_engine_id="seng_8k3m9xr4hjnfg983brhmkhd98n6",
3     overrides={
4         "first_message": True,
5     },
6 )
```

- 2 Then we configure the first message in the client SDK.

React

JavaScript

```
1 conversation.startSession({
2     conversationToken: token,
3     overrides: {
4         agent: {
5             firstMessage: "Hello! How can I help you today?",
6         },
7     },
8 });
```

The first message is spoken by the agent as soon as the connection is established. It does not trigger the `onTranscript` callback on your server - it is handled entirely on the ElevenLabs side.

Next steps

JavaScript SDK reference

Classes, methods, and events for the JavaScript SDK.

Python SDK reference

Classes, methods, and events for the Python SDK.



API reference

Explore all Speech Engine parameters and response formats.



Next.js example app

Run a complete Speech Engine quickstart app locally.

Was this page helpful?



Yes



No

< Previous

Music quickstart

Learn how to generate music with the Music API.

Next >

Built with  **fern**